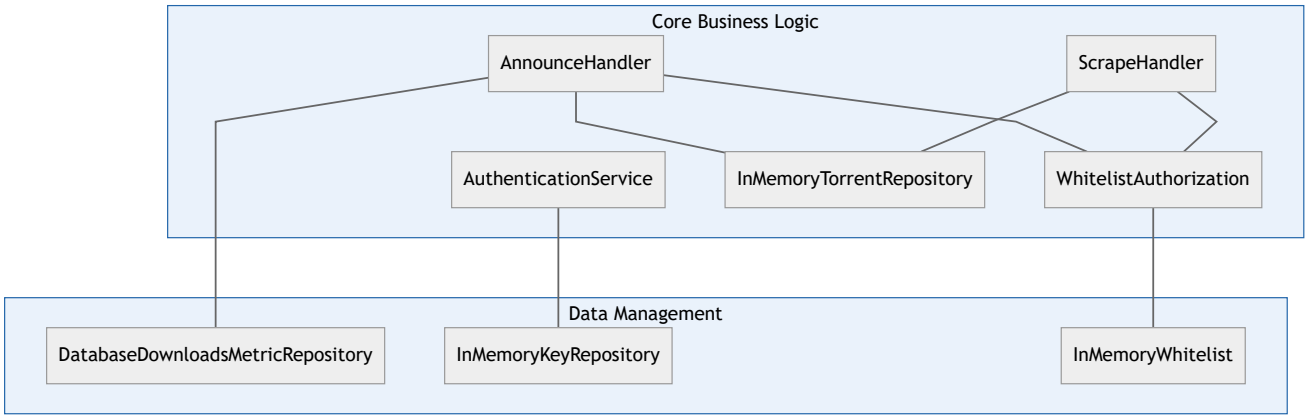
05. 트래커 코어 비즈니스 로직 분석

5.1 개요

Torrust Tracker의 코어 비즈니스 로직은 `packages/tracker-core` 패키지에 구현되어 있으며, BitTorrent 프로토콜의 핵심 기능인 피어 관리, 토렌트 추적, 인증, 그리고 화이트리스트 관리를 담당합니다. 이 모듈은 전송 계층(UDP/HTTP)과 독립적으로 설계되어 있어 다양한 프로토콜에서 재사용할 수 있습니다.

주요 컴포넌트



5.2 AnnounceHandler - 피어 등록 및 관리 로직

5.2.1 AnnounceHandler 구조

위치: `packages/tracker-core/src/announce_handler.rs`

주요 역할:

- 피어의 announce 요청 처리
- 피어 IP 주소 할당 및 검증
- 스웜 메타데이터 관리
- 피어 목록 반환

```
pub struct AnnounceHandler {
/// 트래커 설정
config: Core,
/// 화이트리스트 인증 서비스
whitelist_authorization: Arc<WhitelistAuthorization>,
/// 인메모리 토렌트 저장소
in_memory_torrent_repository: Arc<InMemoryTorrentRepository>,
/// 데이터베이스 다운로드 메트릭 저장소
db_downloads_metric_repository: Arc<DatabaseDownloadsMetricRepository>,
}
```

5.2.2 핵심 메서드: `handle_announcement()`

시그니처:

```
pub async fn handle_announcement(
&self,
info_hash: &InfoHash,
peer: &mut peer::Peer,
remote_client_ip: &IpAddr,
```

```
peers_wanted: &PeersWanted,

) -> Result<AnnounceData, AnnounceError>
```

동작 과정:

1. **화이트리스트 인증 확인**

```
self.whitelist_authorization.authorize(info_hash).await?;
```

2. **피어 IP 주소 할당**

```
peer.change_ip(&assign_ip_address_to_peer(remote_client_ip, self.config.net.exte
```

3. **피어 정보 업데이트**

```
self.in_memory_torrent_repository

.handle_announcement(info_hash, peer, self.load_downloads_metric_if_needed(info_

.await;
```

4. **응답 데이터 구성**

```
Ok(self.build_announce_data(info_hash, peer, peers_wanted).await)
```

5.2.3 IP 주소 할당 로직

함수명: `assign_ip_address_to_peer()`

위치: `packages/tracker-core/src/announce_handler.rs`

```
fn  assign_ip_address_to_peer(remote_client_ip: &IpAddr, tracker_external_ip: Op

if  let Some(host_ip) = tracker_external_ip.filter(|_| remote_client_ip.is_loopb

host_ip

} else {

*remote_client_ip

}

}
```

역할:

- 클라이언트 IP가 루프백 주소인 경우 트래커의 외부 IP로 대체
- 일반적인 경우 클라이언트의 실제 IP 주소 사용

5.2.4 PeersWanted 열거형

```
#[derive(Clone, Debug, PartialEq, Default)]

pub  enum PeersWanted {

#[default]

AsManyAsPossible,

Only { amount: usize },

}
```

기능:

- 클라이언트가 요청하는 피어 수 제한
- 기본값: 최대한 많은 피어 반환 (74개 제한)

- 특정 수량 요청 가능

5.3 ScrapeHandler - 토렌트 통계 처리 로직

5.3.1 ScrapeHandler 구조

위치: `packages/tracker-core/src/scrape_handler.rs`

```
pub struct ScrapeHandler {
/// 화이트리스트 인증 서비스
whitelist_authorization: Arc<whitelist::authorization::WhitelistAuthorization>,
/// 인메모리 토렌트 저장소
in_memory_torrent_repository: Arc<InMemoryTorrentRepository>,
}
```

5.3.2 핵심 메서드: `handle_scrape()`

시그니처:

```
pub async fn handle_scrape(&self, info_hashes: &Vec<InfoHash>) -> Result<Scra
```

동작 과정:

1. **빈 스크랩 데이터 초기화**

```
let mut scrape_data = ScrapeData::empty();
```

2. **각 토렌트에 대한 처리**

```
for info_hash in info_hashes {
let swarm_metadata = match self.whitelist_authorization.authorize(info_hash).a
Ok(()) => {
self.in_memory_torrent_repository
.get_swarm_metadata_or_default(info_hash)
.await
}
Err(_) => SwarmMetadata::zeroed(),
};
scrape_data.add_file(info_hash, swarm_metadata);
}
```

특징:

- 화이트리스트에 없는 토렌트는 0으로 채워진 메타데이터 반환
- 최대 74개 토렌트까지 한 번에 스크랩 가능 (BEP 15 제한)

5.4 피어 생명주기 관리

5.4.1 InMemoryTorrentRepository

위치: `packages/tracker-core/src/torrent/repository/in_memory.rs`

주요 기능:

- 피어 등록, 업데이트, 제거
- 스웜 메타데이터 계산

- 비활성 피어 정리

5.4.2 피어 등록 과정

함수명: `handle_announcement()`

```
pub  async  fn  handle_announcement(
&self,
info_hash: &InfoHash,
peer: &peer::Peer,
opt_persistent_torrent: Option<NumberOfDownloads>,
) {
self.swarms
.handle_announcement(info_hash, peer, opt_persistent_torrent)
.await
.expect("Failed to upsert the peer in swarms");
}
```

처리 과정:

1. 기존 피어인지 확인
2. 새 피어인 경우 스웜에 추가
3. 기존 피어인 경우 정보 업데이트
4. 스웜 통계 재계산

5.4.3 비활성 피어 제거

함수명: `remove_inactive_peers()`

```
pub(crate) async fn remove_inactive_peers(&self, current_cutoff: DurationSince
self.swarms
.remove_inactive_peers(current_cutoff)
.await
.expect("Failed to remove inactive peers from swarms");
}
```

동작:

- 설정된 시간(`inactive_peer_cleanup_interval`) 이후 비활성 피어 제거
- 기본값: 600초 (10분)

5.4.4 피어리스 토렌트 제거

함수명: `remove_peerless_torrents()`

```
pub(crate) async fn remove_peerless_torrents(&self, policy: &TrackerPolicy) {
self.swarms
.remove_peerless_torrents(policy)
.await
.expect("Failed to remove peerless torrents from swarms");
}
```

조건:

- `remove_peerless_torrents` 설정이 `true` 인 경우

- 활성 피어가 없는 토렌트 제거

5.5 스웜 메타데이터 관리 및 통계 계산

5.5.1 SwarmMetadata 구조

```
pub struct SwarmMetadata {
pub complete: u32, // 완료된 활성 피어 수 (시더)
pub downloaded: u32, // 총 완료 다운로드 수
pub incomplete: u32, // 미완료 활성 피어 수 (리처)
}
```

5.5.2 통계 계산 방식

함수명: `get_swarm_metadata_or_default()`

```
pub(crate) async fn get_swarm_metadata_or_default(&self, info_hash: &InfoHash)
self.swarms
.get_swarm_metadata_or_default(info_hash)
.await
.expect("Failed to get swarm metadata")
}
```

계산 로직:

1. **Complete (시더):** `left == 0` 인 활성 피어 수

2. **Incomplete (리처)**: `left > 0` 인 활성 피어 수

3. **Downloaded**: 누적 완료 다운로드 수 (영구 저장)

5.5.3 영구 통계 관리

클래스: `DatabaseDownloadsMetricRepository`

위치: `packages/tracker-core/src/statistics/persisted/downloads.rs`

```
pub struct DatabaseDownloadsMetricRepository {
database: Arc<Box<dyn Database>>,
}
```

주요 메서드:

1. **다운로드 수 증가**

```
pub(crate) fn increase_downloads_for_torrent(&self, info_hash: &InfoHash) -> Re
```

2. **통계 로딩**

```
pub(crate) fn load_torrent_downloads(&self, info_hash: &InfoHash) -> Result<Opt
```

5.6 프라이빗 트래커 모드의 인증 키 관리

5.6.1 AuthenticationService

위치: `packages/tracker-core/src/authentication/service.rs`

```rust
pub struct AuthenticationService {
/// 트래커 설정
config: Core,
/// 인메모리 키 저장소
in_memory_key_repository: Arc<InMemoryKeyRepository>,
}
```

5.6.2 인증 과정

함수명: `authenticate()`

```rust
pub async fn authenticate(&self, key: &Key) -> Result<(), Error> {
if self.tracker_is_private() {
self.verify_auth_key(key).await
} else {
Ok(())
}
}
```

동작 과정:

1. **트래커 모드 확인**

```rust
fn tracker_is_private(&self) -> bool {
self.config.private
```

```
}
```

2. **키 검증**

```
async fn verify_auth_key(&self, key: &Key) -> Result<(), Error> {
match self.in_memory_key_repository.get(key).await {
None => Err(Error::UnableToReadKey { /* ... */ }),
Some(key) => match self.config.private_mode {
Some(private_mode) => {
if private_mode.check_keys_expiration {
return key::verify_key_expiration(&key);
}
Ok(())
}
None => key::verify_key_expiration(&key),
}
}
}
```

5.6.3 키 만료 검증

설정 옵션:

- `check_keys_expiration: true` - 키 만료 시간 검증
- `check_keys_expiration: false` - 만료된 키도 허용

키 타입:

- **영구 키**: `valid_until: None`
- **임시 키**: `valid_until: Some(Duration)`

5.7 화이트리스트 기반 토렌트 접근 제어

5.7.1 WhitelistAuthorization

위치: `packages/tracker-core/src/whitelist/authorization.rs`

```
pub struct WhitelistAuthorization {
/// 트래커 설정
config: Core,
/// 인메모리 화이트리스트
in_memory_whitelist: Arc<InMemoryWhitelist>,
}
```

5.7.2 인증 로직

함수명: `authorize()`

```
pub async fn authorize(&self, info_hash: &InfoHash) -> Result<(), WhitelistEr
if !self.is_listed() {
return Ok(());
}

if self.is_info_hash_whitelisted(info_hash).await {
```

```
return Ok(());
}

Err(WhitelistError::TorrentNotWhitelisted {
info_hash: *info_hash,
location: Location::caller(),
})
}
```

동작 방식:

1. **Listed 모드 확인**

```
fn is_listed(&self) -> bool {
self.config.listed
}
```

2. **화이트리스트 검증**

```
async fn is_info_hash_whitelisted(&self, info_hash: &InfoHash) -> bool {
self.in_memory_whitelist.contains(info_hash).await
}
```

5.7.3 트래커 모드별 동작

| 모드 | 설정 | 동작 |

|------|------|------|

| Public | `listed: false` | 모든 토렌트 허용 |

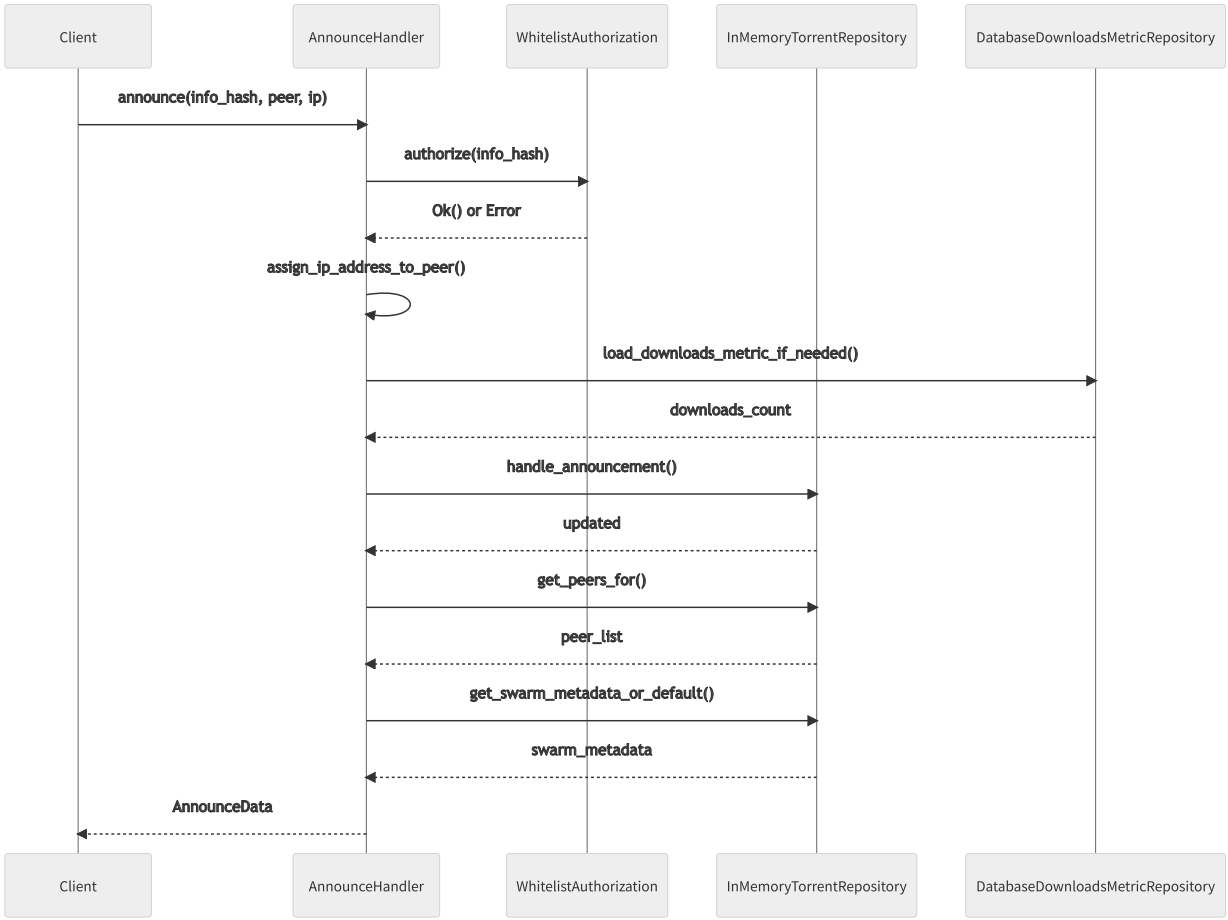
| Listed | `listed: true` | 화이트리스트 토렌트만 허용 |

Listed 모드에서의 처리:

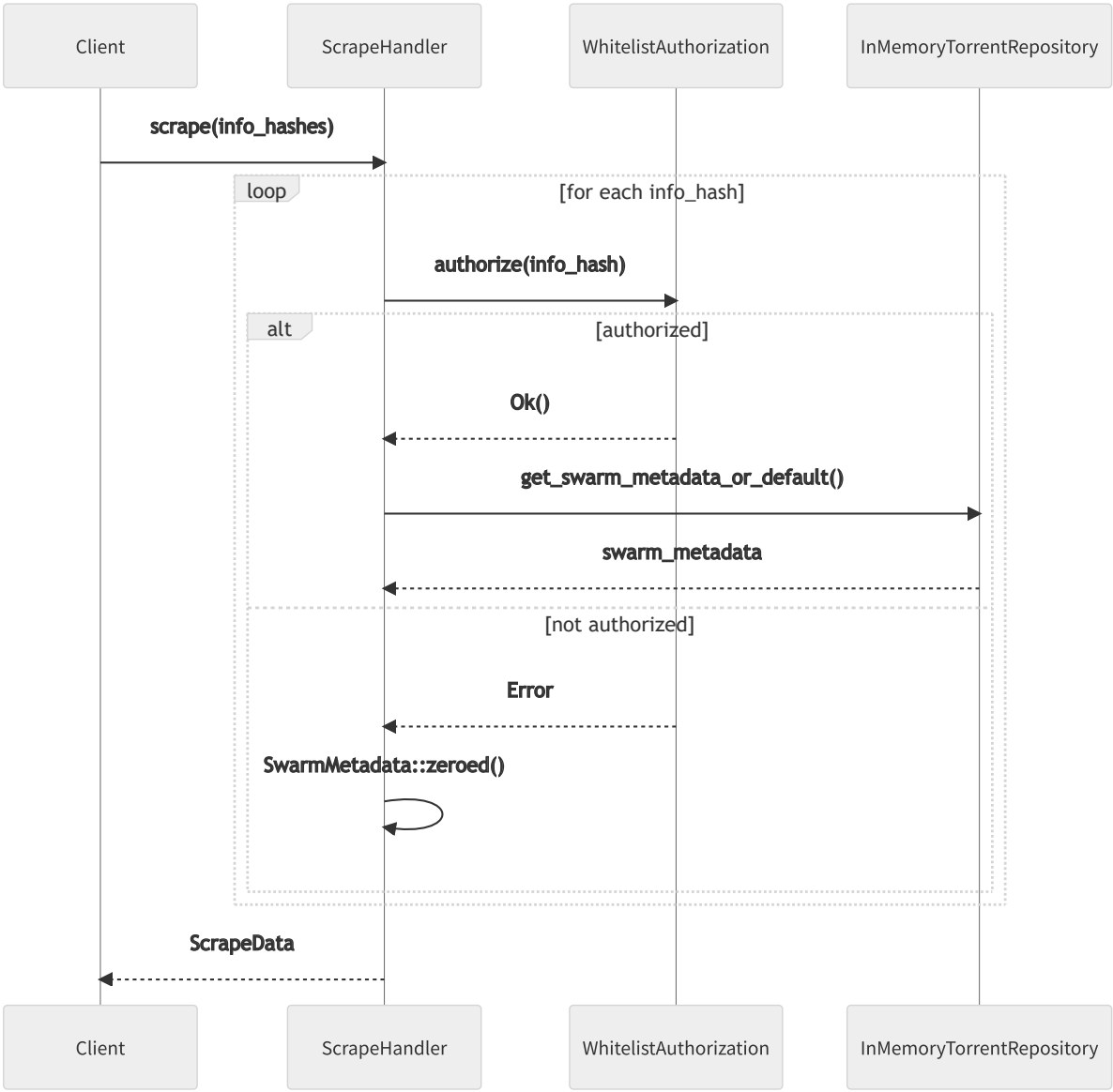
- 화이트리스트에 없는 토렌트: `WhitelistError::TorrentNotWhitelisted` 반환
- Scrape 요청 시: 0으로 채워진 메타데이터 반환

5.8 비즈니스 로직 통합 흐름

5.8.1 Announce 요청 처리 흐름



5.8.2 Scrape 요청 처리 흐름



5.9 성능 최적화 및 메모리 관리

5.9.1 피어 제한

상수: `TORRENT_PEERS_LIMIT = 74`

적용 위치:

- Announce 응답의 최대 피어 수
- Scrape 응답의 최대 토렌트 수 (BEP 15 제한)

5.9.2 백그라운드 정리 작업

설정 옵션:

```
[core]
inactive_peer_cleanup_interval = 600 # 10분
remove_peerless_torrents = true
```

정리 작업:

1. 비활성 피어 제거 (10분 간격)
2. 피어가 없는 토렌트 제거
3. 만료된 인증 키 제거

5.9.3 메모리 효율성

데이터 구조:

- `Arc<peer::Peer>` : 피어 데이터 공유 참조
- `Registry` : 고성능 스웜 데이터 구조
- 인메모리 캐시와 데이터베이스 영구 저장 분리

이러한 비즈니스 로직 구조를 통해 Torrust Tracker는 효율적이고 확장 가능한 BitTorrent 트래커 서비스를 제공하며, 다양한 운영 모드(공개/비공개/화이트리스트)를 지원합니다.